

IoT Development Team, Car Automation Internship
Let Us Support Private Limited

Final Internship Report

Real-Vehicle Data Acquisition, CAN Bus Investigation, and Automated
Acceleration Planning

Internship duration: 29 June 2026 to 29 July 2026

Report date: 31 July 2026

Prepared by the IoT Development Team

Shahid Mihransha, Sapthagiri, Sathish, Shaahir Ahamed, Venkateshwaran, Tumati Pavan

Repository: github.com/IoT-Internship-CarAutomation-LUS/Car-Automation

Executive Summary

Over five weeks, the team designed, built and proved a complete system for acquiring live data from a real vehicle, and produced a costed, safety-first plan for controlling that vehicle's accelerator under electronic command. The central technical result was demonstrated on a real car on 18 July: live engine data read over the OBD-II port and live GPS position captured together on a Raspberry Pi, packed into a fixed 32-byte format, logged locally, and streamed over the network to a separate machine that decoded and verified every packet.

The main outcomes are summarised below.

- A complete real-vehicle data pipeline, proven end to end. 285 packets captured across two sessions, with every packet passing its integrity check.
- A formal, documented data standard, now at version 2.0.0, defining the 32-byte vehicle packet and its decode rules, with an explicit rule that no data field may be claimed as working without a logged response from the real car.
- A deployed backend and two web dashboards, connected live, with a shared message schema and version stamping so the hardware, server and interfaces cannot silently drift apart.
- A fully governed public code repository with branch protection, reviewed pull requests, and an automated daily activity log.
- A documented finding that the vehicle cannot be accelerated electronically through the OBD-II port, and a complete, costed method for doing it physically instead, at roughly 2,600 to 3,350 rupees, without modifying the vehicle in any way.
- An honest record of what remains open, principally the recovery of brake, clutch and air-conditioning signals from the vehicle's CAN bus, and validation of the pipeline with the vehicle in motion.

This report sets out the work in the order it was carried out, records the problems encountered and how each was resolved, and closes with a clear list of next steps for the phase that follows.

Contents

Executive Summary

1. Introduction and Project Objectives
 2. Team and Roles
 3. Project Timeline
 4. System Architecture: Dashboards, Backend and Data Schema
 5. Repository and Engineering Governance
 6. Real-Vehicle Data Acquisition
 - 6.1 Hardware Bring-Up
 - 6.2 The Vehicle Data Acquisition Standard
 - 6.3 Stage 1: Bluetooth ELM327 and Laptop
 - 6.4 Stage 2: Moving to the Raspberry Pi
 - 6.5 Milestone: End-to-End Pipeline Proven on a Real Vehicle
 - 6.6 CAN Bus Investigation for Brake, Clutch and AC
 7. Acceleration and Deceleration: Automated Actuation Planning
 - 7.1 Why Electronic Control via OBD and CAN Was Ruled Out
 - 7.2 Chosen Method: Physical Pedal Actuation
 - 7.3 Safety Measures and Test Sequence
 - 7.4 Cost
 - 7.5 Braking: Scoped for a Later Phase
 8. Challenges Faced and How We Solved Them
 9. Deliverables Produced
 10. Current Status Summary
 11. Next Steps and Future Scope
 12. Conclusion
- Appendix A: Vehicle Data Packet Reference
- Appendix B: Parts and Cost Summary
- Appendix C: Additional Project Photographs

1. Introduction and Project Objectives

This report closes out a five-week Car Automation internship carried out by a six-person IoT Development Team at Let Us Support Private Limited, Chennai. The internship ran from 29 June to 29 July 2026, and this report was finalised on 31 July, once the last planning document was complete. It brings together everything built, tested, learned and decided over that period, drawn from the team's working documents, the project code repository, and the notes kept throughout.

The aspirational end goal set for the project was to take a regular, non-autonomous car and, through research and development, move it toward self-driving capability, climbing the levels of vehicle autonomy as far as is genuinely achievable rather than treating full self-driving as a required deliverable. Two tracks were originally planned to work toward this: a standardised way of acquiring real vehicle data, and a self-built model car demonstrating autonomous navigation with path planning and collision avoidance.

Partway through the internship, the plan was reviewed and the model-car track was set aside, and the whole team was redirected onto real-vehicle data acquisition. This turned out to be the right call for the time available. It let the team go deep on one real, physical system, an actual car, rather than splitting effort across a self-built platform as well. Everything from that point on, including the acceleration and deceleration planning covered later in this report, is a continuation of that single track.

The objectives this report covers, in the order they were tackled, are:

- Build the supporting software: two web dashboards, a backend, and a shared data schema, so the team had somewhere for real data to go before any hardware existed.
- Set up proper engineering governance: branch protection, pull-request review, and an automated daily activity log, so the project stayed organised and auditable across five weeks of parallel work.
- Design a standardised, documented format for acquiring, formatting and streaming vehicle data.
- Prove that standard end to end on a real vehicle, with live engine data and GPS position read, packed, logged and streamed with verified data integrity.
- Investigate the vehicle's CAN bus directly, to recover signals such as brake, clutch and air-conditioning status that are not exposed as standard OBD-II parameters.
- Scope and plan how the vehicle could be accelerated, and later braked, under electronic control, in a way that is safe and does not modify the vehicle.

2. Team and Roles

The team was six people, each owning a clear part of the system. The areas below reflect what each person mainly worked on. Everyone contributed throughout the internship, and work was distributed so that every part of the system had a clear owner who saw it through from design to working result.

Table 1. *The team and the areas each person worked on.*

Person	What they worked on
Shahid Mihransha	Overall planning and sequencing of the project. Hardware bring-up and Raspberry Pi setup, real-world GPS and OBD testing on the vehicle, the data acquisition standard and packet design, repository setup and governance, and coordination of the team's documents and deliverables across both phases.
Sapthagiri	Dashboard 1, the Telemetry Monitor. Built and extended the read-only monitor: drivetrain and engine-health panels, the GPS map with breadcrumb trail, per-panel data freshness indicators, the brake percentage display, and binding the whole interface to the shared message schema.
Sathish	Dashboard 2, the Vehicle Control Console. Built the two-way console: drive, stop and brake controls, the emergency stop, the target-speed slider, the history view, the live feedback panels, and the command message formats sent back through the backend.
Shaahir Ahamed	Backend and streaming API. Built and deployed the WebSocket relay and storage layer, the REST history endpoints, the mock sender scripts that allowed hardware-free testing, the gear estimation algorithm, and the CAN sniffer sketch used in the reverse engineering work.
Venkateshwaran	Hardware and procurement. Parts research and supplier comparison, sourcing the OBD, GPS, CAN and TPMS hardware, managing purchase records and tax invoicing with vendors, and preparing the itemised parts lists used for budget approval.
Tumati Pavan	Real-car sensing and documentation. ELM327 and TPMS research, the sensor and parameter reference work that fed into the acquisition standard, support during the on-site vehicle capture sessions, and project documentation throughout.

3. Project Timeline

Formal day-by-day written reports were kept for roughly the first week of the internship, while the system was being designed and the software foundations built. After that point the team was fully hands-on at the vehicle on site, working directly on the car to get real data, and documents were produced whenever there was genuine progress to report rather than on a fixed daily schedule. The table below summarises the whole period.

Table 2. *Summary of the internship period by phase.*

Period	Focus	Main outcome
29 June to 2 July	Hardware bring-up and system design	GPS live position fix obtained, CAN loopback test passing, system architecture and message schema defined.
3 July to 6 July	Backend, dashboards, repository governance	Backend built and deployed, both dashboards live and connected, repository governed and documentation consolidated.
7 July to 9 July	Data standard and procurement	Vehicle Data Acquisition Standard written, test vehicle selected, acquisition hardware ordered.
10 July to 14 July	First real-vehicle OBD reads	Live engine data read and correctly decoded on the vehicle, acquisition pipeline landed in the repository.
15 July to 18 July	Raspberry Pi, GPS, end-to-end capture	Full pipeline proven on the car: 285 packets captured with 100 percent integrity.
19 July to 29 July	On-site vehicle work and actuation scoping	CAN bus investigation carried out and findings recorded, accelerator actuation method scoped and documented.

4. System Architecture: Dashboards, Backend and Data Schema

Before any real hardware was involved, the team built the software side of the system against fake, schema-shaped data, so that the backend and both dashboards could be fully developed and tested with no sensors present. This meant hardware and software work could proceed in parallel instead of one waiting on the other.

4.1 The two dashboards

- Dashboard 1, the Telemetry Monitor, deployed at `lus.nalusa.space`: a read-only monitor showing drivetrain and dynamics data such as speed, RPM, gear, clutch, brake and fuel, alongside tyre and cabin data, engine health, and a live GPS map with a breadcrumb trail. Per-panel freshness indicators grey out any value that has stopped updating.
- Dashboard 2, the Vehicle Control Console, deployed at `dashboard2.nalusa.space`: a two-way console with drive, stop and brake controls, a target-speed slider, a large emergency stop, and live feedback panels. This dashboard was built for the self-built platform track. When that track was set aside and the project moved to a single focus on the real vehicle, the console and its command messages were retired from the active system.

4.2 The backend

The backend is a self-hosted Python FastAPI and SQLite service, deployed at `api.nalusa.space`, with a single WebSocket endpoint and four REST endpoints covering health checks, latest telemetry, telemetry history and the GPS track. Every message carries a type field, so one endpoint can route incoming messages without needing a separate endpoint per client. Messages are saved to the database and sent live to every connected dashboard at once, which is what allows multiple devices to view the same data simultaneously.

One item remains open at the close of the internship. The original direction was to eventually stream through the company's own cloud infrastructure and a managed database rather than a self-hosted instance. What has been built and proven is the self-hosted version. Migrating the same design onto company infrastructure is recorded as an open item, not yet started, pending provider and credential details. This is a change of hosting only, not a change to the data format.

4.3 The shared data schema

Both dashboards and the backend work from one identical JSON message schema, so nothing drifts between what the hardware sends, what the backend stores, and what a dashboard expects to display. The schema originally defined three message types. Following the move to a single track, version 2.0.0 of the schema removed the two that belonged to the self-built platform, leaving one active type, telemetry, carrying a type field and a millisecond timestamp. A schema version field is stamped on every message, so a dashboard can flag a mismatch instead of silently breaking if the format ever changes. The full byte-level packet layout this schema is built on is given in Appendix A.

5. Repository and Engineering Governance

Alongside the feature work, the team set up and maintained a properly governed public code repository, structured with one folder per area of work, a root readme with a live status board, and a contributing guide describing the branch, pull request and review workflow. With six people working in parallel for five weeks, this was what kept the work organised and reviewable rather than scattered.

5.1 Branch protection and the daily activity log

A rule on the main branch requires pull requests before merging and blocks direct or forced pushes. An automated documentation system reads each day's commits, grouped by folder, into a dated report, with a manual section below it for each person to record what worked, what did not, and the result. This split matters, because version control can show what changed but not whether it actually worked. The system was independently checked before being trusted for daily use, which caught and fixed two real bugs: a date-boundary miscalculation, and a history edge case that could silently drop a commit from a report.

5.2 Keeping the repository clean and honest

Two governance issues came up and were resolved directly rather than worked around. First, branch protection rules do not enforce on a private repository on the free plan, so this was solved by making the repository public, after first confirming there was nothing sensitive in it. Second, once the pull-request rule was properly enforced, the automated daily-log system's own commits started being rejected by that same rule. This was solved with an authorised administrator bypass, since the plan in use did not offer the automation itself as a selectable exception.

A separate cleanup was also carried out after one internal document was found to briefly reference a specific individual's name inside the public repository. A full search of the repository and its history located every occurrence, and each was fixed according to its file type: editable text files were corrected in place and the old versions purged from history, while a document that existed only as a fixed PDF was regenerated with the reference removed, since a PDF's text cannot be found and replaced directly. Database files and other files that should never have been version-controlled were also purged from the entire history, and an ignore rule was added so this cannot happen again.

6. Real-Vehicle Data Acquisition

This is the heart of the internship's technical work. The goal was simple to state and hard to get right end to end: read live data from a real car's OBD-II port and from a GPS module, pack it into a small, fixed binary format, and stream it somewhere useful, with no step of that chain silently misreporting what it actually measured.

6.1 Hardware Bring-Up

Early hardware bring-up proved each sensor path on its own before anything was combined. The NEO-6M GPS module was wired over a serial connection and gave a genuine live position fix in Chennai, with six to seven satellites and a good accuracy figure. The MCP2515 CAN module was proven over its SPI connection using a loopback self-test, in which the module sends a frame to itself and receives it back. This confirms the wiring and the chip itself, though, as later investigation showed, it does not confirm anything about a live vehicle bus. A SIM800L mobile module was brought up alongside these but did not respond reliably and was set aside for a later session.

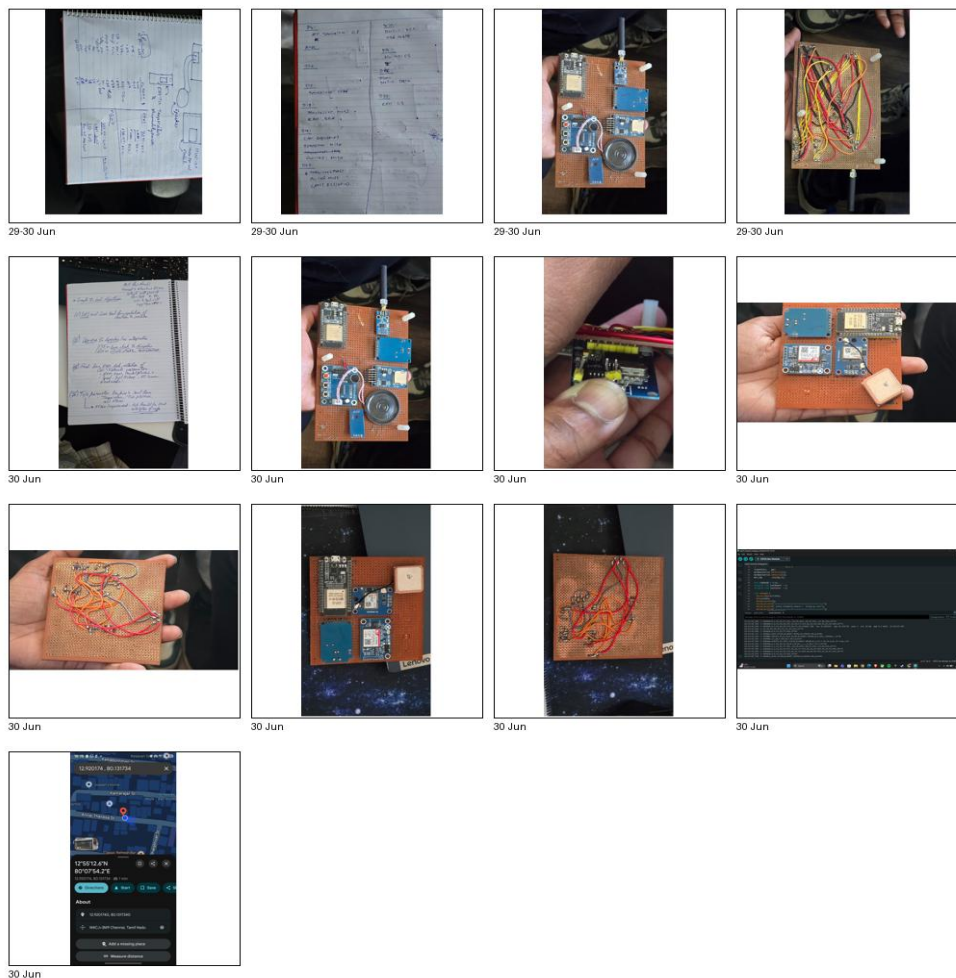


Figure 1. Hardware bring-up on 29 and 30 June: the GPS, CAN and mobile modules laid out and wired before combined testing.

A recurring early lesson concerned power. The Raspberry Pi 4 that later became the main acquisition device needs a proper 5 volt, 3 amp supply. Powering it from a laptop's USB port is fine for initial setup but risks silent undervoltage in the field, which shows up as random

resets or corrupted storage rather than as an obvious error. A further, easy-to-miss issue was found here too: some high-wattage USB-C cables contain a chip that, on a Raspberry Pi 4, causes the board to draw no power at all. Once identified, cable selection accounted for this, and the Pi was fitted with a case, heatsink and fan, which brought operating temperatures down substantially.

6.2 The Vehicle Data Acquisition Standard

Before writing acquisition code, the team wrote a formal standard as the single source of truth for how data is obtained, formatted, stored and streamed. It defines a 32-byte binary vehicle packet as the wire format between hardware and the backend, with the backend unpacking it into readable JSON exactly once before anything reaches a dashboard. A separate 16-byte tyre packet was also designed, though the tyre sensors were never purchased, so that part of the design was never wired to hardware. Keeping the wire format small and fixed is what makes the data cheap to transmit and simple to verify.

Every field in the standard is tagged with one of four categories: a standard parameter expected to work on virtually any compliant car; a value that exists in the standard but may be manufacturer-specific or unreliable; a value calculated by the team's own code rather than read from the car; or a value that comes from a separate sensor entirely, such as GPS or the team's own tyre sensors. The standard is explicit that no field may be marked as confirmed working on the basis of an online claim, a datasheet, or an AI-generated answer. Only a logged response from the real car counts. The full byte layout is reproduced in Appendix A.

6.3 Stage 1: Bluetooth ELM327 and Laptop

The first real-car reads were done the simplest possible way: an ELM327 Bluetooth adapter plugged into the OBD-II port, paired to a laptop, with a Python script polling eight target parameters once per second. This adapter draws its power from the car's OBD port and has no power source of its own, so meaningful testing needed either a real car or a bench supply feeding the connector.

On the first real-car session, the adapter connected without trouble and the car replied to every requested parameter. Engine speed stayed around 840 to 860 RPM at idle and rose above 2,000 when the engine was revved, matching what could be heard. Coolant read 88 to 90 degrees Celsius, consistent with a warmed engine. The air-flow reading tracked engine speed, giving a second, independent way to confirm the decoding was genuine and not coincidence. Two things were flagged honestly rather than hidden: vehicle speed could not yet be validated, since the car was parked throughout, and fuel level returned a negative reply on every read, which is simply how this particular vehicle is built and was correctly recorded as unavailable rather than as zero.

Getting the acquisition script robust for a real car took real debugging. The first request after protocol auto-detection can come back prefixed with the word `SEARCHING`, which the early script did not strip, silently discarding a perfectly good reading. A response-validation check was added that only accepts replies beginning with a valid acknowledgement and matching the parameter actually requested, and returns a clear null result for anything else, whether that is `SEARCHING`, `NO DATA`, `UNABLE TO CONNECT`, or a truncated fragment. A second, more fundamental fix was needed too. The whole polling loop had originally lived inside the network connection, so a network failure before the car was ever reached would prevent any data, even the local backup file, from being captured at all. The capture loop was restructured

to run entirely on its own, with local logging always active and the network layer strictly best effort on top.

6.4 Stage 2: Moving to the Raspberry Pi

Once the decode logic was proven, the same acquisition script was moved onto the Raspberry Pi 4, with the GPS module wired directly to its serial pins so that position and engine data could be captured by a single device in the car. A lightweight direct network link was also added as a bench-test path, separate from the route to the backend, so that a laptop could act as a stand-in for the cloud endpoint and receive raw, then decoded, packets, with a full integrity check on arrival. Two small receiver programs were built for this: one printing genuinely raw, undecoded data to prove the connection itself works, and one unpacking every packet and flagging any that failed its integrity check rather than trusting corrupted values.

6.5 Milestone: End-to-End Pipeline Proven on a Real Vehicle

On 18 July, the team proved the complete pipeline running together on the vehicle for the first time. The Raspberry Pi read live engine data over Bluetooth and live GPS position at the same time, combined both into the standard 32-byte packet, logged every packet locally, and forwarded each one over the network to a laptop receiver standing in for the cloud.

```

lus-pi@LUS-PI: ~ - ssh - Car-Automat
crw-rw---- 1 root dialout 216, 0 Jul 18 16:36 /dev/rfcomm0
lus-pi@LUS-PI:~/.Car-Automation/backend $ python3 elm327_bt.py --raw
=====
ELM327 Bluetooth OBD-II Acquisition Script
COM Port : /dev/rfcomm0 | Baud: 38400
Log Dir : /home/lus-pi/Car-Automation/logs
Backend : DISABLED (offline mode)
TCP      : DISABLED
Modes    : stream=False, tcp=False, raw=True, fast=False
=====
[LOG] Session logs -> /home/lus-pi/Car-Automation/logs
[LOG] Session 163712 started.
[GPS] Opened /dev/serial0 at 9600 baud.
[BT] Opened /dev/rfcomm0 at 38400 baud.
[BT] Initialising ELM327...
[BT] ATZ -> OK
[BT] AT+Z -> OK
[BT] AT+L -> OK
[BT] ATSP0 -> OK
[BT] AT+H0 -> OK
[BT] ELM327 initialised successfully.
[OBD] ===== SUPPORTED PIDS (0100) =====
[OBD] SEARCHING...r41 00 8E 3F 88 13'
[OBD] =====
[LOG] Created /home/lus-pi/Car-Automation/logs/2026-07-18/stream/stream_2026-07-18.csv
[OBD] Starting synchronous capture loop (1 cycle per 1.0s)...
[OBD] --raw modifier enabled: printing raw hex bytes
[RAW] 010C -> 41 8C 0D 2E
[LOG] Created /home/lus-pi/Car-Automation/logs/2026-07-18/raw/raw_2026-07-18.csv
[RAW] 018D -> 41 8D 09
[RAW] 0185 -> 41 85 88
[RAW] 0104 -> 41 04 6D
[RAW] 0111 -> 41 11 28
[RAW] 012F -> 7F 01 2F
[RAW] 0110 -> 41 10 01 07
[RAW] 010F -> 41 0F 66
[RAW] 0146 -> 7F 01 46
[RAW] 010B -> 41 0B 33
[RAW] 0101 -> 41 01 00 07 E1 00
[OBD] RPM=303, speed=0km/h, coolant=88C, fuel=None, maf=2.63, batt=13.8V
[GPS] lat=12.885346 lng=79.938221 sats=6 fix=True

```

Figure 2. Initialisation and the first combined engine and GPS reading on the vehicle.

```

lus-pi@LUS-PI:~/Car-Automation$
[OBD] RPM=1027.0 spd=0km/h cool=87C fuel=None% maf=3.49 batt=13.8V
[GPS] Lat=12.805332 lng=79.938207 sats=6 fix=True
[PACKET] 6A 5B 5E 8F 10 0C 00 00 C3 64 D4 04 C3 C2 9E 46 01 50 00 1F 7F FF 8A 00 00 41 65 FF 32 C8 25 6F
[RAW] 010C -> 41 0C 00 AE
[RAW] 010D -> 41 0D 00
[RAW] 0105 -> 41 05 7F
[RAW] 0104 -> 41 04 6D
[RAW] 0111 -> 41 11 28
[RAW] 012F -> 7F 01 2F
[RAW] 0110 -> 41 10 01 0A
[RAW] 010F -> 41 0F 65
[RAW] 0146 -> 7F 01 46
[RAW] 0108 -> 41 08 32
[RAW] 0101 -> 41 01 00 07 E1 00
[OBD] RPM=975.5 spd=0km/h cool=87C fuel=None% maf=2.66 batt=13.8V
[GPS] Lat=12.805332 lng=79.938207 sats=6 fix=True
[PACKET] 6A 5B 5E 90 00 AE 00 00 C3 64 D4 04 C3 C2 9F 46 01 0A 00 1F 7F FF 8A 00 00 55 65 FF 32 C8 26 E1
[RAW] 010C -> 41 0C 00 80
[RAW] 010D -> 41 0D 00
[RAW] 0105 -> 41 05 7F
[RAW] 0104 -> 41 04 6D
[RAW] 0111 -> 41 11 28
[RAW] 012F -> 7F 01 2F
[RAW] 0110 -> 41 10 01 0D
[RAW] 010F -> 41 0F 65
[RAW] 0146 -> 7F 01 46
[RAW] 0108 -> 41 08 32
[RAW] 0101 -> 41 01 00 07 E1 00
[OBD] RPM=864.0 spd=0km/h cool=87C fuel=None% maf=2.69 batt=13.8V
[GPS] Lat=12.805333 lng=79.938208 sats=7 fix=True
[PACKET] 6A 5B 5E 91 01 80 00 00 C3 64 D5 04 C3 C2 A0 47 01 0D 00 1F 7F FF 8A 00 00 54 65 FF 32 C8 27 41
[RAW] 010C -> 41 0C 00 70
[RAW] 010D -> 41 0D 00
[RAW] 0105 -> 41 05 7F
[RAW] 0104 -> 41 04 6D
[RAW] 0111 -> 41 11 28
[RAW] 012F -> 7F 01 2F
[RAW] 0110 -> 41 10 01 10
[RAW] 010F -> 41 0F 65
[RAW] 0146 -> 7F 01 46

```

Figure 3. Decoded engine values, GPS position, and the packed 32-byte packet, shown for each one-second cycle.

Across two capture sessions that afternoon, the system logged 285 complete packets, and every single one passed its integrity check. Engine speed ranged from about 688 to 3,126 RPM, including a deliberate throttle press to confirm the readings tracked the engine in real time. The air-flow reading rose and fell together with engine speed as expected. Coolant held between 87 and 90 degrees Celsius, battery voltage read a steady 13.8 volts, and GPS held a fix on every packet with six to seven satellites. Vehicle speed read zero throughout, since the car was kept stationary for this session, and capturing data with the vehicle moving was recorded as the immediate next step.

```

lus-pi@LUS-PI:~/Car-Automation$
remote: Resolving deltas: 100% (5/5), completed with 3 local objects.
To https://github.com/IoT-Internship/CarAutomation-LUS/Car-Automation.git
8720044..3e01a84  gps-integration -> gps-integration
lus-pi@LUS-PI:~/Car-Automation$ scp lus-pi@pi-ip:~/Car-Automation/backend/decoded_receiver.py .
bash: pi-ip: No such file or directory
lus-pi@LUS-PI:~/Car-Automation$ hostname -I
10.20.126.232
lus-pi@LUS-PI:~/Car-Automation$ cd ~/Car-Automation/backend
python3 elm327_bt.py --raw --tcp 10.20.126.107
=====
ELM327 Bluetooth OBD-II Acquisition Script
COM Port : /dev/rfcomm0 | Baud: 38400
Log Dir : /home/lus-pi/Car-Automation/logs
Backend : DISABLED (offline mode)
TCP : 10.20.126.107:9000
Modes : stream=False, tcp=True, raw=True, fast=False
=====
[LOG] Session logs -> /home/lus-pi/Car-Automation/logs
[LOG] Session 164909 started.
[TCP] --tcp enabled. Will forward raw packets best-effort to 10.20.126.107:9000
[GPS] Opened /dev/serial0 at 9600 baud.
[BT] Opened /dev/rfcomm0 at 38400 baud.
[BT] Initialising ELM327...
[BT] ATZ -> OK
[BT] AT+V -> OK
[BT] AT+L -> OK
[BT] ATSP0 -> OK
[BT] AT+H -> OK
[BT] ELM327 initialised successfully.
[OBD] ===== SUPPORTED PIDS (0100) =====
[OBD] *SEARCHING...r41 00 BE 3F 88 13*
[LOG] =====
[LOG] Appending to /home/lus-pi/Car-Automation/logs/2026-07-18/stream/stream_2026-07-18.csv
[OBD] Starting synchronous capture loop (1 cycle per 1.0s)...
[OBD] --raw modifier enabled: printing raw hex bytes
[OBD] --tcp modifier enabled: raw packet forwarding to 10.20.126.107:9000 active
[RAW] 010C -> 41 0C 00 28
[LOG] Appending to /home/lus-pi/Car-Automation/logs/2026-07-18/raw/raw_2026-07-18.csv
[RAW] 010D -> 41 0D 00
[RAW] 0105 -> 41 05 81
[RAW] 0104 -> 41 04 6E

```

Figure 4. The same capture running with live network forwarding enabled, ready to reach a real cloud endpoint over the same path unchanged.

```

C:\WINDOWS\system32\cmd. x + v
C:\Users\shahi>python decoded_receiver.py
[TCP] decoded_receiver listening on 0.0.0.0:9080
[TCP] Waiting for a connection.
[TCP] Connected: 10.20.126.232:58542
RPM=842.0 speed=0 coolant=89 fuel=None lat=12.885345 lng=79.938192 fix=True crc_valid=True
RPM=861.5 speed=0 coolant=89 fuel=None lat=12.885345 lng=79.938192 fix=True crc_valid=True
RPM=840.5 speed=0 coolant=89 fuel=None lat=12.885344 lng=79.938192 fix=True crc_valid=True
RPM=851.5 speed=0 coolant=89 fuel=None lat=12.885344 lng=79.938193 fix=True crc_valid=True
RPM=847.0 speed=0 coolant=89 fuel=None lat=12.885344 lng=79.938193 fix=True crc_valid=True
RPM=838.0 speed=0 coolant=89 fuel=None lat=12.885344 lng=79.938191 fix=True crc_valid=True
RPM=856.0 speed=0 coolant=89 fuel=None lat=12.885344 lng=79.938193 fix=True crc_valid=True
RPM=837.0 speed=0 coolant=89 fuel=None lat=12.885345 lng=79.938193 fix=True crc_valid=True
RPM=850.5 speed=0 coolant=89 fuel=None lat=12.885345 lng=79.938193 fix=True crc_valid=True
RPM=841.0 speed=0 coolant=89 fuel=None lat=12.885346 lng=79.938192 fix=True crc_valid=True
RPM=841.5 speed=0 coolant=89 fuel=None lat=12.885347 lng=79.938192 fix=True crc_valid=True
RPM=861.5 speed=0 coolant=89 fuel=None lat=12.885348 lng=79.938192 fix=True crc_valid=True
RPM=841.0 speed=0 coolant=89 fuel=None lat=12.885349 lng=79.938192 fix=True crc_valid=True
RPM=854.5 speed=0 coolant=89 fuel=None lat=12.88535 lng=79.938191 fix=True crc_valid=True
RPM=845.5 speed=0 coolant=89 fuel=None lat=12.88535 lng=79.938191 fix=True crc_valid=True
RPM=831.0 speed=0 coolant=89 fuel=None lat=12.885351 lng=79.938191 fix=True crc_valid=True
RPM=855.0 speed=0 coolant=89 fuel=None lat=12.885352 lng=79.938191 fix=True crc_valid=True
RPM=838.5 speed=0 coolant=89 fuel=None lat=12.885354 lng=79.938191 fix=True crc_valid=True
RPM=856.5 speed=0 coolant=89 fuel=None lat=12.885355 lng=79.938191 fix=True crc_valid=True
RPM=849.5 speed=0 coolant=89 fuel=None lat=12.885356 lng=79.938191 fix=True crc_valid=True
RPM=857.0 speed=0 coolant=89 fuel=None lat=12.885358 lng=79.93819 fix=True crc_valid=True
RPM=845.0 speed=0 coolant=89 fuel=None lat=12.885359 lng=79.938191 fix=True crc_valid=True
RPM=850.0 speed=0 coolant=89 fuel=None lat=12.885359 lng=79.938191 fix=True crc_valid=True
RPM=858.5 speed=0 coolant=89 fuel=None lat=12.885359 lng=79.93819 fix=True crc_valid=True
RPM=848.0 speed=0 coolant=89 fuel=None lat=12.885361 lng=79.93819 fix=True crc_valid=True
RPM=851.5 speed=0 coolant=89 fuel=None lat=12.885362 lng=79.938189 fix=True crc_valid=True
RPM=849.0 speed=0 coolant=89 fuel=None lat=12.885362 lng=79.938189 fix=True crc_valid=True
RPM=849.5 speed=0 coolant=89 fuel=None lat=12.885364 lng=79.938188 fix=True crc_valid=True
RPM=857.5 speed=0 coolant=89 fuel=None lat=12.885364 lng=79.938187 fix=True crc_valid=True
RPM=849.5 speed=0 coolant=89 fuel=None lat=12.885366 lng=79.938186 fix=True crc_valid=True
RPM=926.0 speed=0 coolant=89 fuel=None lat=12.885368 lng=79.938185 fix=True crc_valid=True
RPM=935.0 speed=0 coolant=89 fuel=None lat=12.885369 lng=79.938184 fix=True crc_valid=True
RPM=840.0 speed=0 coolant=89 fuel=None lat=12.885369 lng=79.938184 fix=True crc_valid=True
RPM=845.0 speed=0 coolant=89 fuel=None lat=12.885369 lng=79.938184 fix=True crc_valid=True
RPM=833.0 speed=0 coolant=89 fuel=None lat=12.88537 lng=79.938183 fix=True crc_valid=True

```

Figure 5. Received on the laptop standing in for the cloud: every packet decoded and passing its integrity check. This was the first time the full path from car to a separate machine was shown working end to end.

Two of the eight requested values came back marked unsupported on every cycle: fuel level and ambient air temperature. The car answers these with a standard not-supported code rather than a value, which the software correctly records as blank rather than as a fabricated zero. This is a genuine characteristic of this vehicle, not a fault in the system, and recording it truthfully is exactly what the standard requires.

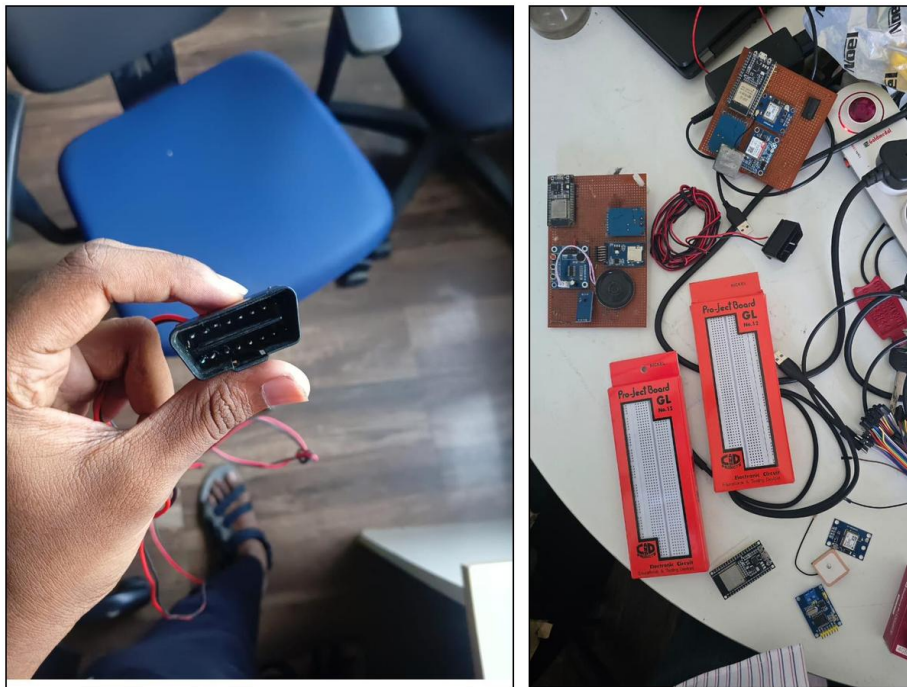


Figure 6. On-site work at the vehicle. Left: live capture inside the car on 10 July with the engine running and data streaming into the terminal. Right: the assembled in-car hardware on 14 July, including the Raspberry Pi, microcontroller boards, GPS and CAN modules.

6.6 CAN Bus Investigation for Brake, Clutch and AC

Brake pedal status, clutch position, and air-conditioning state are not exposed as standard parameters on this vehicle. Recovering them requires a different method: connecting a CAN module to the car's data bus in listen-only mode, and identifying which message on the bus

changes when a control, such as the brake pedal, is operated. This is the one part of the project where a mistake can talk back to the vehicle's own electronics, so it was approached carefully, and every capture was done in a mode where the hardware is physically incapable of transmitting.

Two separate approaches were tried:

- A CAN module with a microcontroller, in listen-only mode. This is the intended production path, but bring-up surfaced several real hardware issues: the module's 5 volt transceiver risks feeding 5 volts back into a 3.3 volt controller's pins unless correctly level-shifted; the module's crystal frequency must match the software configuration exactly, or the effective data rate is silently halved and no valid frames are ever received; the car's bus is already terminated at both ends, so the module's own termination link has to be removed; and a shared ground reference between the vehicle connector and the sniffing hardware proved to be one of the two most likely causes of a persistent zero-frames result, since the signal is measured relative to the receiver's own ground.
- The adapter's own built-in monitoring mode, read and correlated with a purpose-built terminal tool added to the repository, which highlights in place exactly which bytes on which message identifier change when a control is operated. This removes the need for a separate CAN module during the discovery phase.

A genuinely important lesson came out of this stage. The loopback self-test that had already passed during hardware bring-up proves nothing about a live vehicle bus. Loopback is internal to the chip: both the sending and receiving sides are wrong by the same factor if the crystal setting is wrong, so they still agree with each other perfectly. A passing loopback test shows the wiring and the chip are alive. It says nothing about the transceiver, the data rate against a real bus, or the ground reference being correct on an actual car. This is worth recording because it is exactly the kind of test result that can give false confidence.

One question was not conclusively resolved by the close of the internship: whether this vehicle's OBD-II port carries body-network traffic such as brake and air-conditioning signals at all, or whether the factory gateway only passes powertrain and diagnostic traffic to that port and blocks the rest. The direct test for this, logging the bus for thirty seconds with nothing touched while the engine runs, was identified but not completed in time. If the port turns out to be isolated in this way, no amount of software fixes it from the OBD-II connector, and the practical fallback identified is a direct hardware tap, for example on the brake-light switch feed for brake status. That is a small, low-cost addition rather than a blocker to the rest of the project.

7. Acceleration and Deceleration: Automated Actuation Planning

With the data-acquisition pipeline proven, the team scoped the next phase of the project making the car accelerate under automated control. This section also records how the later, separate task of braking has been scoped, without being started.

7.1 Why Electronic Control via OBD and CAN Was Ruled Out

The first route considered was electronic: sending an acceleration command through the OBD-II port, or injecting a signal onto the CAN bus that imitates the accelerator pedal. This was investigated and found not to be workable. The OBD-II port is a read-only diagnostic interface and does not accept commands that drive the throttle, and the engine control unit is deliberately designed to reject external or spoofed pedal signals for safety reasons. The car cannot be driven this way from outside electronics, by design rather than by accident. This finding was reported

on 18 July, together with an open invitation for any documentation or contacts that might allow the electronic route to be re-checked before committing fully to a physical alternative.

7.2 Chosen Method: Physical Pedal Actuation

The method chosen, and the one the team plans to take forward, is physical: a high-torque servo motor mounted on a small frame in the footwell, with an arm that presses the accelerator pedal down by a controlled amount when the servo rotates, and lets the pedal's own return spring bring it back to idle when the servo returns. A microcontroller, connected to a laptop, commands the servo's position.

Three deliberate simplifications keep this design safe and easy to remove. The servo only pushes and is never attached to the pedal, so no clamps or linkage are needed. The rig rests in the footwell and braces against the driver's seat base, which is already bolted to the floor, so the reaction force of pressing the pedal is taken safely, and nothing on the vehicle is drilled, cut or bolted. The whole assembly lifts out in under a minute. The servo also holds its own position internally, so no complex control software is required at this stage. Importantly, this approach works with the vehicle's own safety systems rather than against them. The engine control unit sees a completely normal pedal press, and a person in the driver's seat can override the rig at any time simply by using the pedals normally.



Figure 7. *Footwell of the test vehicle, showing the floor-hinged accelerator pedal on the right. The arm would contact the upper part of the pedal face, where there is the most leverage.*

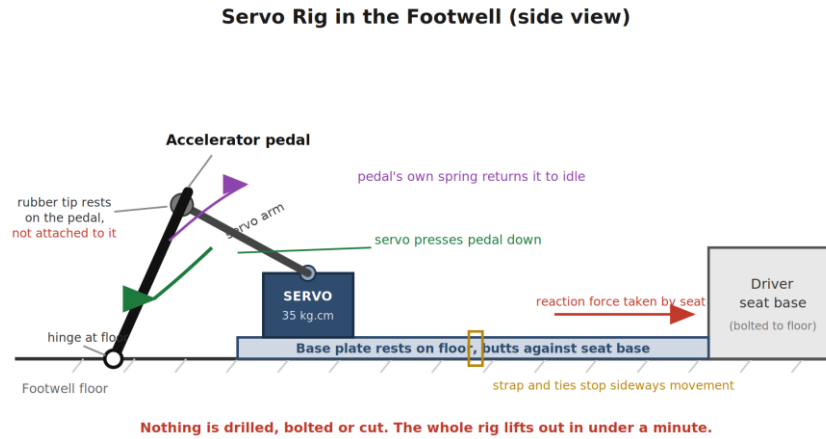


Figure8. How the servo rig would be arranged in the footwell, viewed from the side. Nothing is drilled, bolted or cut.

Power is taken from the car's 12 volt accessory socket, which is already fused by the vehicle and requires no modification, and stepped down through a converter to the servo's rated voltage, with a capacitor across the supply to absorb the current surges the servo draws when it moves. The controller is powered separately, from the laptop, so it stays in command even if the vehicle supply dips. The controller moves the commanded pedal position gradually rather than jumping straight to it, with software limits on both the maximum position and the maximum rate of change, so no command can apply full throttle instantly.

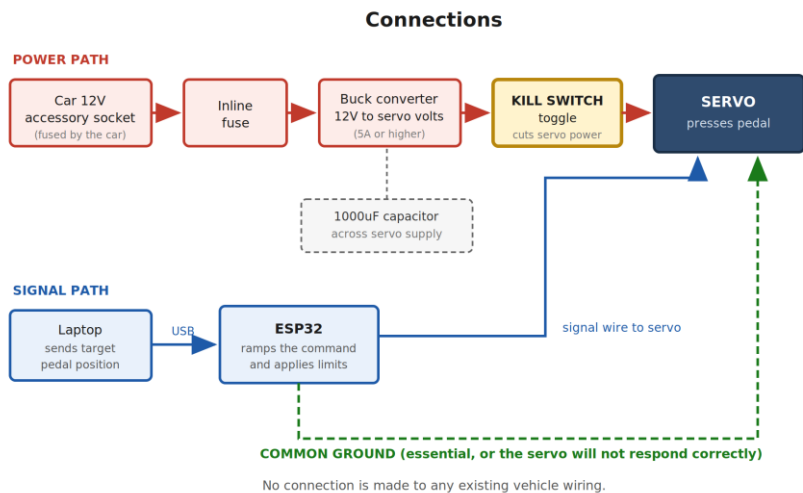


Figure9. Connections in block form: power path in red, signal path in blue, common ground in green. No connection is made to any existing vehicle wiring.

Verification of this stage does not need any new sensors. The data pipeline already built and proven in Section 6 is used directly. When a pedal position is commanded, the throttle position and engine speed read back from the car are logged, showing directly whether the command produced the intended result. This is a useful outcome in itself: the earlier phase of the project becomes the measuring instrument for the next one.

7.3 Safety Measures and Test Sequence

Several independent layers of safety are built into the plan, deliberately overlapping so that no single failure removes control of the vehicle:

- A hardware toggle kill switch in the servo's power line. Flipping it removes power, and the pedal's own spring returns it to idle, with no software involved.
- Unplugging the accessory socket plug, or switching off the ignition, both of which also remove power.
- Software limits on the maximum pedal position and the maximum rate of change.
- The driver can override the rig at any time using the pedals normally, and the clutch can be pressed to disconnect drive.
- All testing is done with the vehicle stationary, in neutral, handbrake applied, in an open area, and under company supervision.

The planned test sequence proceeds in stages, each proving one thing before the next is attempted:

- Measure the actual pedal force with a simple hook scale, to confirm the chosen servo size is adequate before committing to a part.
- Assemble and test the rig on a desk, away from the vehicle, checking smooth movement and confirming the kill switch works.
- Fit the rig in the vehicle with the engine off, confirming the arm presses the pedal through its full range, the rig stays firmly in place, and it never obstructs the driver.
- Start the engine, remain stationary in neutral with the handbrake applied, and command a range of pedal positions while logging throttle position and engine speed.
- Review the logged data to confirm the response is smooth, repeatable, and stays within the programmed limits.

7.4 Cost

The estimated cost for a first accelerator prototype using this method is small, at roughly 2,600 to 3,350 rupees. The first item on the list is deliberate: measuring the actual pedal force before buying anything means the servo is chosen against a measured figure rather than an assumption, so the right part is bought once.

Table 3. Indicative parts and costs for a first accelerator actuation prototype.

Item	Purpose	Approx. cost (Rs.)
Hook-type luggage scale	Measure actual pedal force before buying a servo	200
High-torque metal-gear servo, 35 kg cm class	Presses the accelerator pedal	1,200 to 1,800
Aluminium servo horn and rubber tip	Arm that contacts the pedal face	150
12 volt accessory socket plug with lead	Takes power from the car	150
Step-down converter, 12 volt in, 5 amp or more	Supplies the servo at its rated voltage	250 to 400
Heavy-duty toggle switch	Hardware kill switch	100
Inline fuse holder and fuses	Protection	80
1000 microfarad capacitor	Stabilises the servo supply	20
Base plate and angle section	Rig frame in the footwell	300
Straps, zip ties, fasteners	Securing the rig and cable routing	150
Microcontroller and jumper wires	Controller, already in stock	in stock

The vehicle currently used for testing has a manual gearbox. Controlling the accelerator alone is enough to prove throttle actuation, which is the aim of this phase, but is not sufficient on its own to make a manual car move, since that also needs coordinated clutch and gear control, a considerably larger task. On an automatic vehicle, the same rig described here would be sufficient to produce controlled forward movement on its own, and would reach a moving demonstration significantly faster. With the vehicle the team has been working with, physical accelerator actuation is what has been found practical, and is the direction the team plans to take.

7.5 Braking: Scoped for a Later Phase

Brake actuation is deliberately kept as a separate, later task, and no brake parts have been ordered. It needs substantially greater force and a stricter safety design than accelerator actuation, including a quick-release link that separates under load, and is estimated in the region of 8,000 to 15,000 rupees or more. All actuation testing, including any future brake work, is planned for an open area under company supervision, with the vehicle in neutral and the handbrake applied to begin with, and with hardware and software kill switches in place before anything is allowed to drive a pedal.

8. Challenges Faced and How We Solved Them

A number of real problems came up across the internship. Recording them honestly, and how each was actually resolved, is as much a part of this report as the successes, and several of these findings are worth carrying into any future work on the same vehicle.

8.1 Hardware and power

- Powering the Raspberry Pi from a laptop's USB port risked silent undervoltage, which appears as random resets rather than a clear error. Resolved by sourcing the correct 5 volt, 3 amp supply for real sessions, with a power bank used only as an interim measure.
- A high-wattage USB-C cable with a built-in chip caused the Pi to draw no power at all, a known quirk of that board. Identified and avoided by checking cables before use.
- The Pi ran hot under sustained use in the car. Resolved by fitting a case with a heatsink and fan, which reduced temperatures substantially.

8.2 Software and data integrity

- The first reply after protocol auto-detection could come back prefixed with the word SEARCHING, which the decoder did not expect and would mishandle, throwing away a valid reading. Fixed with a response-validation check that only accepts a properly formed reply and returns a clear null result for anything else.
- The original capture script made the entire polling loop dependent on a live network connection, so a network failure before ever reaching the car would prevent any data, even the local backup file, from being captured. Restructured so capture runs entirely on its own and always works, with networking as a best-effort layer on top.
- An unsupported reading, such as fuel level on this vehicle, risked being packed and reported as a fabricated zero rather than a genuine null, which would have quietly

broken the standard's own rule of marking missing data as unavailable. Fixed so that a value which decodes to null stays null all the way through packing, unpacking, and the data sent to the dashboards.

8.3 CAN bus reverse engineering

- A CAN module's crystal frequency must match its software configuration exactly, or the effective data rate is silently halved and no valid frames are received, with no error message to indicate why.
- A passing loopback self-test was found to prove nothing about a live vehicle bus, since loopback is internal to the chip and does not exercise the transceiver, the data rate against a real bus, or the ground reference, all three of which matter once a real car is involved.
- A mismatched or missing shared ground reference between the vehicle connector and the sniffing hardware was identified as one of the two most likely causes of a persistent zero-frames result on a real vehicle.
- Whether this vehicle's port carries body-network traffic at all, or whether a factory gateway isolates it, remains an open question at the close of the internship. This was flagged honestly rather than assumed either way, with a low-cost hardware-tap fallback identified in case the answer turns out to be no.

8.4 Repository and documentation integrity

- Branch protection rules do not enforce on a private repository on the free plan. Resolved by making the repository public, after first confirming there was nothing sensitive in it.
- Once pull-request enforcement was active, the automated daily-log system's own commits started being rejected by the same rule it needed to bypass. Resolved with an authorised administrator exception.
- A pull request that claimed to contain bug fixes was merged, but the file content on the main branch still had the original bugs, because an older, unfixed version had been committed by mistake. Caught by checking the repository's actual file content rather than trusting the commit messages, and fixed by rewriting the affected files from verified, tested content.
- An internal document was found to briefly reference a specific individual's name inside the public repository. Both the editable files and a fixed PDF copy were corrected, the PDF by regenerating it since its text cannot be found and replaced directly, and the old versions were purged from history.
- A branch created before a history rewrite no longer shared a common history with the rewritten main branch. A plain merge would have attached a large amount of disconnected parallel history, so this was resolved by replaying the branch's full content as a single new commit on top of the current main instead.

9. Deliverables Produced

The following were produced and handed over during the internship. All code and documentation is held in the project repository.

9.1 Working systems

- Vehicle data acquisition pipeline: reads engine data over the OBD-II port and GPS position together, packs them into the standard 32-byte packet, logs locally, and streams over the network. Proven on a real vehicle.
- Backend service: WebSocket relay, storage, and four REST endpoints, deployed and connected to the telemetry dashboard.
- Dashboard 1, the Telemetry Monitor, deployed and live.
- Dashboard 2, the Vehicle Control Console, built and deployed, then retired along with the self-built platform track.
- Receiver programs for verifying raw and decoded packets, including integrity checking.
- A CAN monitoring tool for identifying which bus messages change when a vehicle control is operated.
- Automated daily activity logging, running against the repository.

9.2 Documents

- Vehicle data standard, currently at version 2.0.0, defining the packet format, decode rules, data reliability categories, acquisition procedure and validation method.
- Vehicle selection study, ranking suitable vehicles for data acquisition and setting out why real compliance can only be confirmed by testing.
- Parts lists and hardware comparison documents used for budget approval.
- System design and integration plan, and the reframed project plan.
- Progress reports covering the real-vehicle testing sessions and the end-to-end data collection milestone.
- Accelerator actuation method document, including fitting, wiring, control, safety and costing.
- This final report.

10. Current Status Summary

Table 4. Status of each part of the project at the close of the internship.

Area	Status
Telemetry dashboard	Built and deployed, connected to a live backend. The separate control console was retired with the self-built platform track.
Backend service	Built and deployed, self-hosted. Migration to company cloud infrastructure is an open item, not yet started.
Data standard and message schema	Complete and at version 2.0.0: 32-byte vehicle packet, sentinel values for unavailable readings, validity bits, and version stamping on every message.
Repository governance	Fully governed: branch protection, pull-request workflow, automated daily log, public and clean.
Real-vehicle data acquisition	Proven end to end: 285 packets at 100 percent integrity on 18 July. Moving-vehicle capture is the next validation step.
CAN bus signals: brake, clutch, AC	In progress. Real hardware findings recorded; the gateway question is not yet conclusively resolved.
Acceleration, accelerator actuation	Method chosen, costed and fully documented. Not yet built or bench-tested.
Deceleration, braking	Scoped for a separate, later phase. Not yet started.

11. Next Steps and Future Scope

- Resolve the CAN gateway question with a direct listen-only capture test. If body-network traffic is not reachable at the vehicle's diagnostic port, fall back to a direct hardware tap for brake and air-conditioning status.
- Repeat the data-acquisition capture with the vehicle moving, to validate the speed and gear values, which have only been tested stationary so far.
- Build and bench-test the accelerator actuation rig, then work through the staged test sequence in Section 7.3, from desk test to engine-off fit check to engine-on stationary test, before any supervised low-speed motion is considered.
- Scope and budget the brake actuation phase separately, once accelerator actuation is proven, with the additional force and safety requirements it needs.
- Resolve whether the self-hosted backend satisfies the original direction to use company cloud infrastructure, or whether the same design should be migrated onto it.
- Continue reporting progress as each phase completes, in the same staged-checkpoint style used throughout this internship.

12. Conclusion

Over five weeks, the team went from an ambitious two-track plan to a single, deeply proven real-vehicle data-acquisition system, backed by properly governed software and repository infrastructure, and a clearly scoped, safety-first plan for the next phase of automated control. The project's own honesty rules were followed throughout, including in this report: never claim a data field is confirmed without a logged response from the real car, always flag an open question rather than assume an answer, and record what did not work alongside what did.

The core milestone the internship set out to prove, that live engine and position data can be read from a real car, packed into a standard format, and streamed reliably with verified integrity, was achieved and demonstrated on 18 July. Just as importantly, the work produced findings that will save time for whoever continues it: that the diagnostic port cannot be used to drive the vehicle, that a passing loopback test says nothing about a live bus, and that physical actuation working alongside the car's own safety systems is the practical route forward.

The work that remains, recovering the remaining CAN signals, validating the pipeline in motion, and building the actuation hardware, is clearly scoped, costed and sequenced. It is a natural continuation of the same track rather than a new direction, and the team is in a position to hand it over cleanly.

Appendix A: Vehicle Data Packet Reference

The 32-byte vehicle packet combines engine data, GPS position and device status. The layout below is the one actually running in the acquisition code, used for the capture on 18 July, and it matches version 2.0.0 of the shared message schema held in the project repository. Version 2.0.0 was a deliberate revision of the original 1.0.0 design: it added a timestamp, an estimated gear, manifold pressure, ambient temperature, a device health byte and a sequence counter, replaced the simple checksum with a CRC-8, changed percentage scaling so that a full 100 percent reading can no longer be confused with the not-available marker, and introduced the validity bits described below.

Table 5. *The 32-byte vehicle data packet layout.*

Byte(s)	Field	Decode and notes	Category
0 to 3	Timestamp	Unix seconds	Device
4 to 5	Engine speed (RPM)	value x 4, 0xFFFF if unavailable	Standard
6	Vehicle speed (km/h)	direct value, 0xFF if unavailable	Standard
7 to 10	GPS latitude	signed, degrees x 1,000,000	From GPS
11 to 14	GPS longitude	signed, degrees x 1,000,000	From GPS
15	GPS satellites and fix	bits 0 to 5 satellite count, bit 6 fix flag	From GPS
16 to 17	Air flow, MAF (g/s)	g/s x 100, 0xFFFF if unavailable	May be absent
18	Gear	0 neutral, 1 to 6 gears, 0x0E reverse, 0xFF unknown	Calculated
19	Throttle position (%)	percent x 2, 0xFF if unavailable	Standard
20	Coolant temperature (C)	degrees C + 40, 0xFF if unavailable	Standard
21	Fuel level (%)	percent x 2, 0xFF if unavailable	May be absent
22	Battery voltage (V)	volts x 10, 0xFF if unavailable	Standard
23	Warning lamp and fault count	bit 7 lamp on, bits 0 to 6 fault count	Standard
24	AC, brake and clutch flags	bit-packed with validity bits, see note below	From CAN, pending
25	Engine load (%)	percent x 2, 0xFF if unavailable	Standard
26	Intake air temperature (C)	degrees C + 40, 0xFF if unavailable	Standard
27	Ambient temperature (C)	degrees C + 40, 0xFF if unavailable	May be absent
28	Manifold pressure, MAP (kPa)	kPa direct, 0xFF if unavailable	Standard
29	Device health	bit 7 power, bit 6 GPS, bit 5 CAN	Device
30	Sequence counter	increments 0 to 255, then repeats	Device
31	CRC-8 checksum	CRC-8-CCITT over bytes 0 to 30	Always present

Two details in the table above are worth expanding. First, unavailable values are written as a reserved sentinel rather than as zero: 0xFF for a single byte and 0xFFFF for a two-byte value. These are safe to use because they are unreachable for the fields concerned, so a receiver can always tell a missing reading from a real one. Second, byte 24 carries a state bit and a separate validity bit for each of brake, clutch and air conditioning: bit 0 brake state and bit 1 brake valid, bit 2 clutch state and bit 3 clutch valid, bit 4 air conditioning state and bit 5 valid. A validity bit of zero means the signal has not been found on the CAN bus yet, which is not the same as

the signal being off. This is what stops an unknown value being displayed as a confident reading, and it is why these three fields are shown as pending rather than as false.

A separate 16-byte tyre packet was designed for pressure and temperature from the team's own aftermarket 433 MHz sensors, since most vehicles in this market have no factory tyre pressure system to read from. The sensors were never purchased, so this packet was never wired to hardware and does not form part of the current schema. It is recorded here as a completed design, ready to be picked up if tyre sensing is taken forward.

Table 6. *The 16-byte tyre data packet layout.*

Byte(s)	Field	Encoding
0 to 1	Front-left pressure	value / 10 = kPa
2 to 3	Front-left temperature	value / 10 = degrees C
4 to 5	Front-right pressure	value / 10 = kPa
6 to 7	Front-right temperature	value / 10 = degrees C
8 to 9	Rear-left pressure	value / 10 = kPa
10 to 11	Rear-left temperature	value / 10 = degrees C
12 to 13	Rear-right pressure	value / 10 = kPa
14 to 15	Rear-right temperature	value / 10 = degrees C

Appendix B: Parts and Cost Summary

A consolidated view of hardware costs across the project. Figures are planning estimates, confirmed against actual invoices at the time of purchase.

Table 7. *Consolidated hardware cost summary.*

Category	Approx. cost (Rs.)
Core boards: microcontrollers, GPS, CAN modules, bring-up phase	1,650 to 2,300
OBD-II acquisition: adapter, extension cable, interface	1,900 to 2,900
Tyre pressure sensing: sensors and receiver	2,150 to 2,800
Wiring and miscellaneous	about 240
Accelerator actuation prototype, see Section 7.4	2,600 to 3,350
Brake actuation, later phase, not yet ordered	8,000 to 15,000 or more

Appendix C: Additional Project Photographs

A further selection of photographs from across the internship is included below for a complete record of the work.

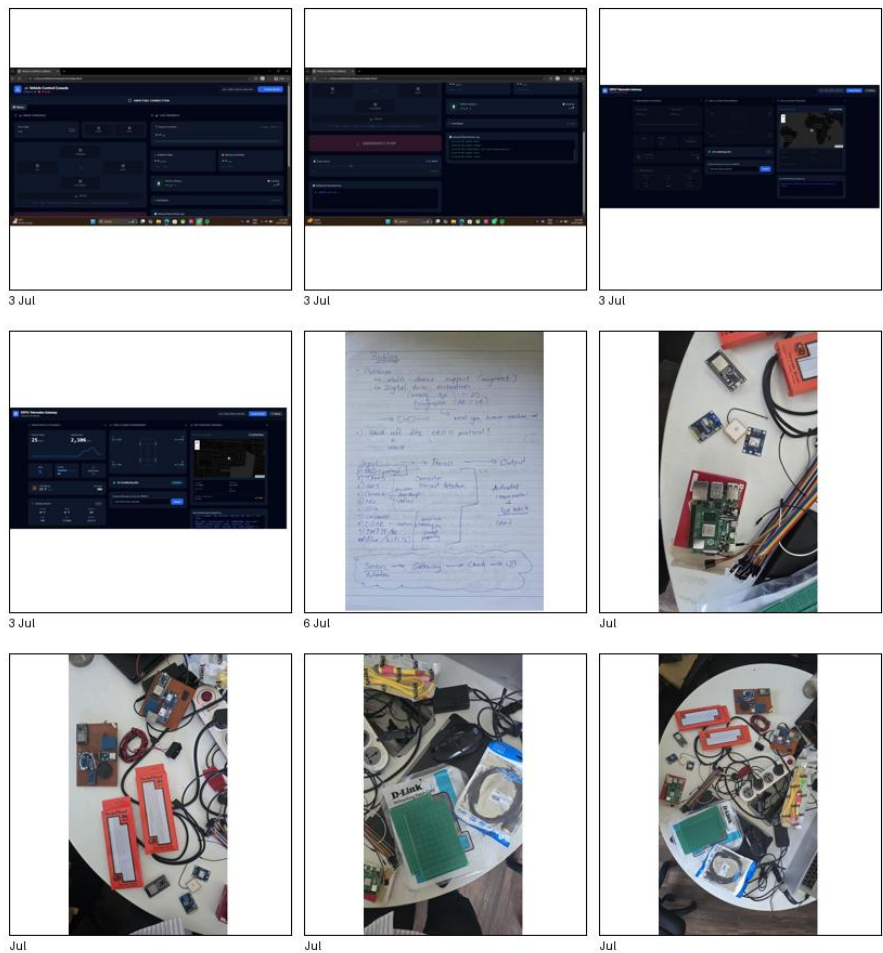


Figure 10. Additional project photographs: dashboard and hardware testing sessions from early July, and further hardware and on-site testing photographs from later in the internship.